UNIVERSIDAD NACIONAL DE SAN AGUSTIN

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS

Laboratorio**: Introducción a Qt**

I. OBJETIVOS

- Introducción al lenguaje Qt.

II. TEMAS A TRATAR

- Funciones básicas QT
- Clase QObject
- Comunicación entre objetos QT por Signal & Slots

III. MARCO TEORICO

Signals & Slots

- SEÑAL es un mensaje enviado por un objeto, es la respuesta a un evento.
- SLOT O RANURA: es una función que se llamará cuando se active una "señal".
- La función de CONEXIÓN especifica qué señal está vinculada con qué ranura/slot.

Qt tiene implementado en su SDK señales y slots para ser utilizadas.
Los objetos como: Botones, checkbox, grillas, etc tienen sus señales.

Ejemplos:
QPushButton su señal es el evento clicked()
La aplicación en general que hereda de la clase QApplication tiene el slot quit() (terminar la aplicación), se lanza o activa cuando le damos clic en salir o terminamos la aplicación.

Nomenclatura de una conexión:
connect(sender, &Sender::signalName, receiver, &Receiver::slotName)

IV. ACTIVIDADES

(IDE QT Creator)

a) Funciones primitivas Qt y la clase QObject

Se deben de agregar las librerías, QDebug para colocar semáforos y poder realizar el debugging, y el QObject para implementar la herencia.

```
#include <QDebug>
#include <QObject>
```

Introducción a datos y funciones cadena Qt

```
int x=47; double y=3.2478;

char t = 'Ok';      multi-c


QString qtcadena = QString("%1 %2 %3").arg(t).arg(y).arg(x);

string cadena = qtcadena.toStdString(); //convertir dato primitivo string a QString
qDebug() << cadena.c_str();  //convertir para mostrar en el debug
qDebug() << qtcadena;

qtcadena = "arequipa ciudad blanca 2021";
qDebug() << qtcadena.contains("ciudad");    //devuelve bool
qDebug() << qtcadena.indexOf("2021");       //devuelve la posición.
```

b) Usando la clase QOBJECT del SDK Qt, para controlar el uso de memoria.

```
class adulto : public QObject
{
private:
    QString nombre;

public:
    adulto(QObject *base = 0): QObject(base){

    }
    void setNombre(const QString &nomb){
        nombre = nomb;
    }
    QString getNombre() const{
        return nombre;
    }
    ~adulto(){
        qDebug() << "..objeto destruido...";
    }
};
```

Implementación en el main(). La sentencia “delete ” invoca al QObject para destruir todos los objetos hijos y clase base.

```
adulto *papa = new adulto;
adulto *hijo1 = new adulto(papa);
adulto *hijo2 = new adulto(papa);
hijo1->setNombre("manuel");
hijo2->setNombre("Jose");

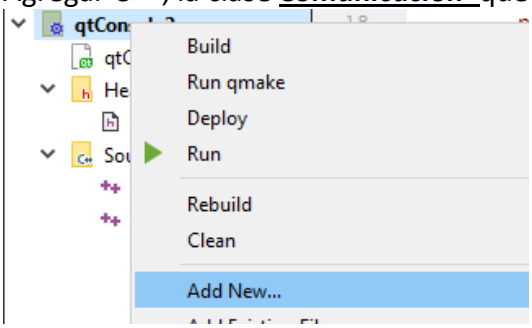
delete papa;
```

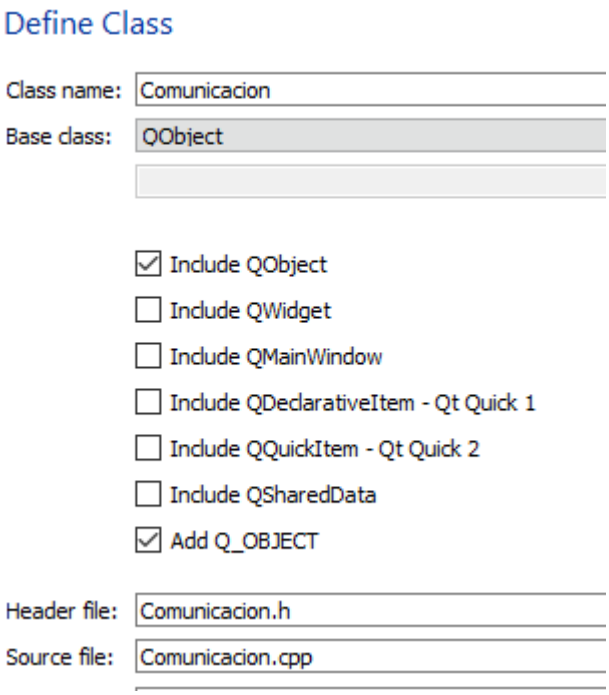
La salida del debugging debe mostrar tres eliminaciones automáticas de objetos.

```
07:03:10: Starting D:\n_
..objeto destruido...
..objeto destruido...
..objeto destruido...
```

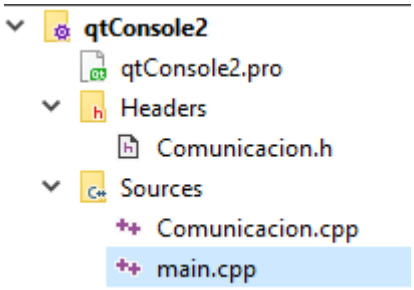
c) usando Signals & Slots, implementar un chat, comunicación entre 2 objetos persona.

En el actual proyecto Qt Aplication -> console.
Agregar C++, la clase ***Comunicación*** que herede de la clase QObject





El resultado debe ser: Se crean los archivos Comunicacion.h y el .cpp



Editar el archivo Comunicación.h

```
#ifndef COMUNICACION_H
#define COMUNICACION_H

#include <QObject>

class Comunicacion : public QObject
{
    Q_OBJECT
public:
    explicit Comunicacion(QObject *parent = nullptr);

    void setNombre(const QString &nomb){
        strnombre=nomb;
    }

    void conversar(const QString &conversa);


signals:
    void enviarmensaje(QString);

public slots:
    void escuchar(const QString &str);

private:
    QString strnombre;
};

#endif // COMUNICACION_H
```

metodos de la clase
Hereda de Object

la señal en respuesta a un evento

canal físico donde se realiza
la acción

Editar el archivo Comunicación.cpp

```
#include "Comunicacion.h"
#include <QDebug>

Comunicacion::Comunicacion(QObject *parent) : QObject(parent)
{

}

void Comunicacion::escuchar(const QString &str){

    qDebug() << "estoy escuchando...."<<str;
}

void Comunicacion::conversar(const QString &conversa){

    //emitir la señal
    emit enviarmensaje(conversa);
}
```

implementar el slot y el metodo que invocará la señal del evento

Implementar en la función principal la clase base (padre) y sus hijas, y crear la conexión de las señales y slots.

```
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    QObject *person = new QObject;
    Comunicacion *pepe = new Comunicacion(person);
    Comunicacion *jose = new Comunicacion(person);
    //set....

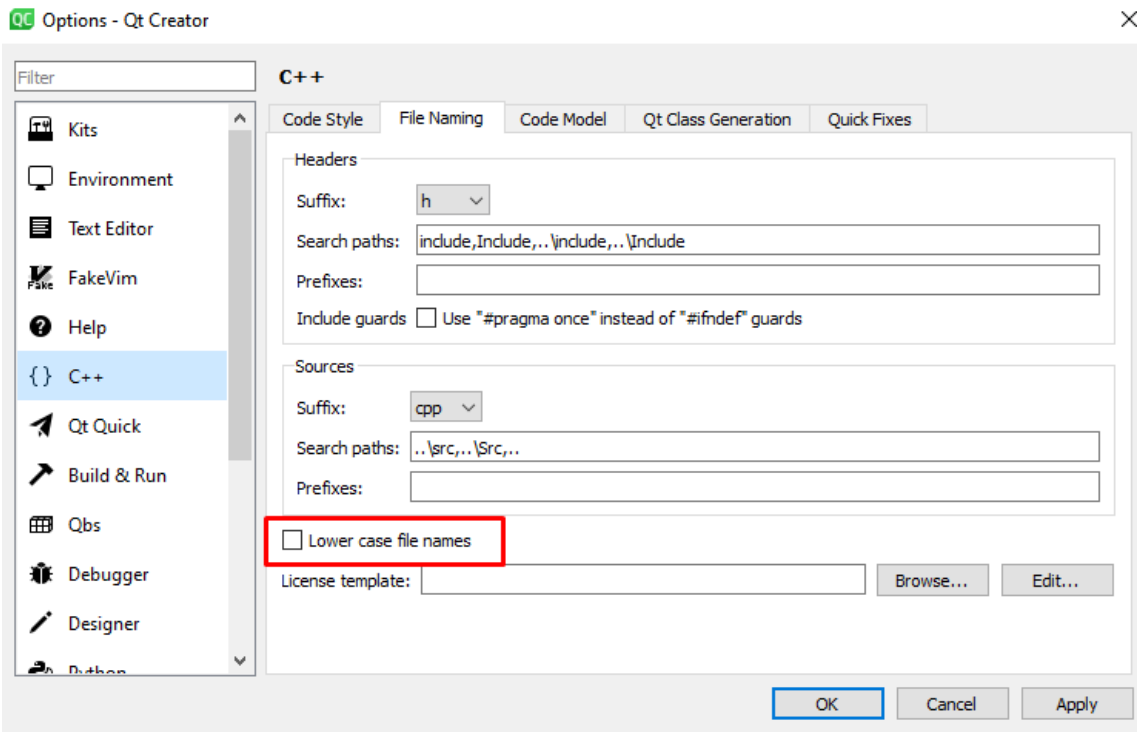
    QObject::connect(pepe,SIGNAL(enviarmensaje(QString)),jose,SLOT(escuchar(QString)));
    QObject::connect(jose,SIGNAL(enviarmensaje(QString)),pepe,SLOT(escuchar(QString)));
    pepe->conversar("buenos dias");
    jose->conversar("como vas?");

    delete person;
```

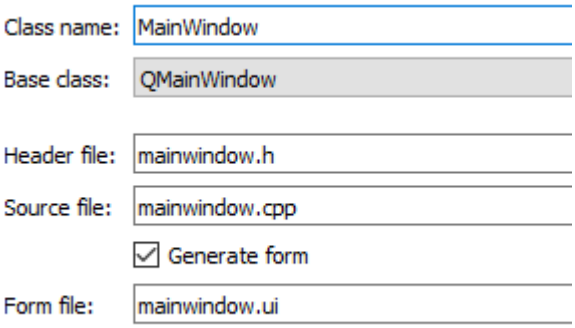
Ejecutar y observar la comunicación.

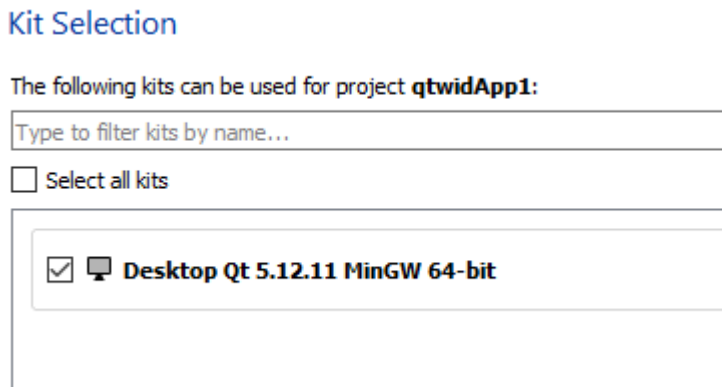
d) <u>Aplicación Qt Widgets, signals & slots en objetos de formulario GUI</u>

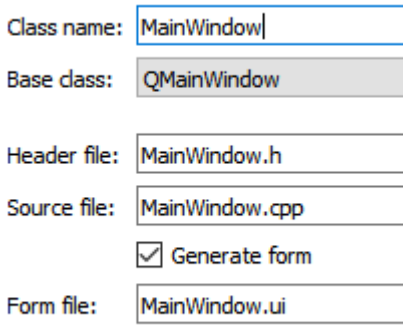
(IDE QT Creator)
Previamente desactivar el uso de minúsculas para escribir nombre de clases con mayusculas.


Options - Qt Creator
Filter
Kits
Environment
Text Editor
FakeVim
Help
C++
Qt Quick
Build & Run
Qbs
Debugger
Designer
C++
Code Style
File Naming
Code Model
Qt Class Generation
Quick Fixes
Headers
Suffix:
h
Search paths:
include,Include,..\include,..\Include
Prefixes:
Include guards
Use "#pragma once" instead of "#ifndef" guards
Sources
Suffix:
cpp
Search paths:
..\src,..\Src,..
Prefixes:
Lower case file names
License template:
Browse...
Edit...
OK
Cancel
Apply

Ahora, crear
File → New File or Project → Application → Qt Widgets Application (escoger Sistema qmake).
Dejar todo por defecto.



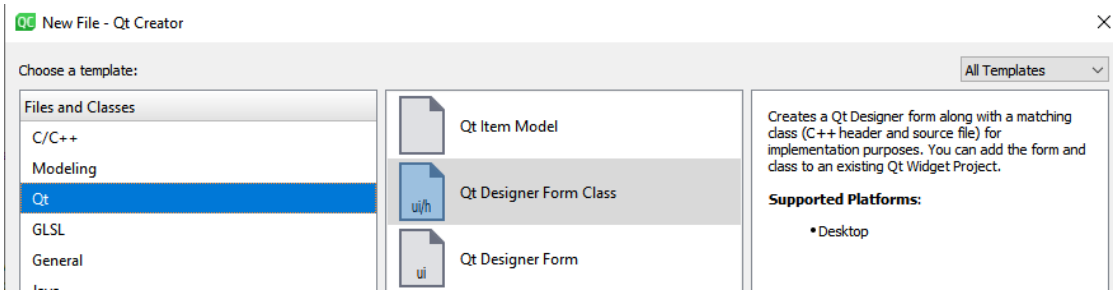


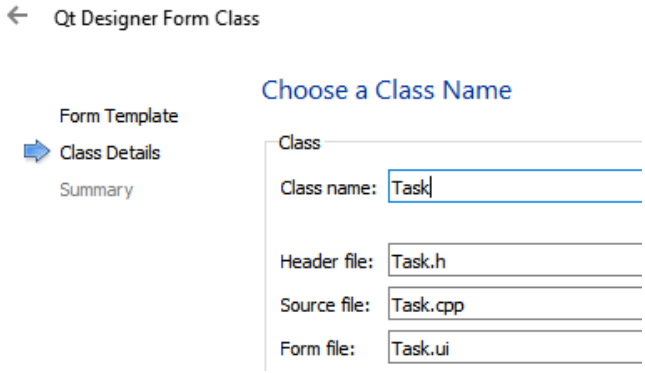


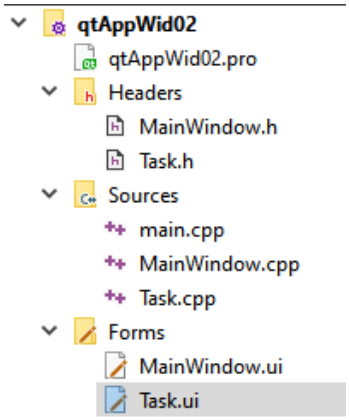
Editar el archivo MainWindow.ui

Agregar los siguientes objetos:
Label → statusLabel propiedad text: Status
Button → addTaskButton propiedad text: Agregar tarea

Agregar la clase TASK del tipo “Qt Dedigner form class” y template WIDGET.







En el formulario Task.ui agregar los objetos
checkbox (objectName = checkbox)
text (objectName = Mitext)

Notas:

- MainWindow.h contiene la estructura de la clase MainWindow, que hereda de la super clase QMainWindow (SDK del Qt).
  El constructor debe ser explicito, y debe inicializar un puntero en NULL de tipo QWidget (responsable del GUI)
- En Qt, QApplication es una clase y puede ser instanciada como cualquier clase, esta es parte del entorno de la aplicación y tiene sus slot por defecto (Exit o Quit).
- MainWindows.ui → contiene 2 objetos, un checkbox y un Botton (push buttom). Estos objetos debemos direccionar funcionalidad por signal y slot.
- *ui es un puntero del tipo MainWindows, servirá para instanciar objetos del formulario ui.

- MainWindow.cpp implementa las signals y Slots. En el constructor podemos implementar la conexión entre los objetos "ui" con las señales y slots.
  Para nuestro caso, se requiere personalizar la llamada del clic (signal) del botón del formulario hacia un slot particular (por ahora el slot debe escribir en la consola debugging).
- "delete ui", siempre debe ir en el destructor del CPP, al estar bajo el contexto del Q_OBJECT se destruyen automáticamente las instancias de las clases hijas y base de la memoria.

1.- Inicialmente, conectaremos el botón a la instancia de la aplicación que esta tomando control por defecto:
En el constructor del MainWindow.cpp

```
connect(ui->addTaskButton, &QPushButton::clicked, QApplication::instance(), &QApplication::quit)
```

addtaskButton es el boton que esta en el formulario.

Compilar y ejecutar. Cuando damos clic en el botón la aplicación hace Quit (es el slot).

2.- Agregar un nuevo slot personalizado, en el MainWindow.h se debe agregar:

```
public slots:
    void addTask();
```

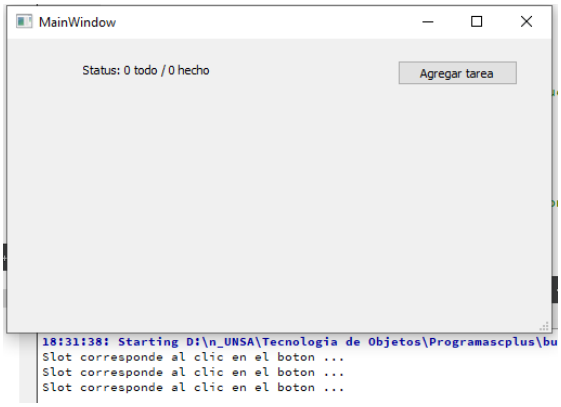
Ahora implementar en el CPP como un método y que registre una cadena en el debugging.

```
qDebug() << "Slot corresponde al clic en el boton ...";
```

3.- Una vez declarado el slot e implementado, se debe reformular la conexión en el constructor.

```
connect(ui->addTaskButton, &QPushButton::clicked, this, &MainWindow::addTask)
```

4.- Ahora, ejecutar la aplicación y, hacer clic en el botón y debe escribir en la consola.



VII. Ejercicios

1. Crear una interface gráfica (que implemente señales y slots) que muestre una lista de nombres de colores, al dar clic sobre alguno que se muestre un Label o un Text con el nombre del color.

2. Averiguar la implementación por signals y Slots, para comunicar los objetos del formulario del Task.ui (por ejemplo una caja de texto, o un Label) desde el MainWindows.h y CPP.

Referencias:

- https://www.qt.io/
- QT creator manual